

Lawrence Berkeley National Laboratory

Lawrence Berkeley National Laboratory

Title

New Security Results on Encrypted Key Exchange

Permalink

<https://escholarship.org/uc/item/6p17j026>

Authors

Bresson, Emmanuel
Chevassut, Olivier
Pointcheval, David

Publication Date

2003-12-15

New Security Results on Encrypted Key Exchange

Emmanuel Bresson¹, Olivier Chevassut², and David Pointcheval³

¹ Dépt Cryptologie, CELAR, 35174 Bruz Cedex, France
Emmanuel.Bresson@polytechnique.org.

² Lawrence Berkeley National Laboratory, Berkeley, CA 94720, USA,
OChevassut@lbl.gov – <http://www.itg.lbl.gov/~chevassu>.

³ Dépt d’informatique, École normale supérieure, 75230 Paris Cedex 05, France
David.Pointcheval@ens.fr – <http://www.di.ens.fr/users/pointche>.

Abstract. Schemes for *encrypted key exchange* are designed to provide two entities communicating over a public network, and sharing a (short) password only, with a session key to be used to achieve data integrity and/or message confidentiality. An example of a very efficient and “elegant” scheme for *encrypted key exchange* considered for standardization by the IEEE P1363 Standard working group is AuthA. This scheme was conjectured secure when the symmetric-encryption primitive is instantiated via either a cipher that closely behaves like an “ideal cipher”, or a mask generation function that is the product of the message with a hash of the password. While the security of this scheme in the former case has been recently proven, the latter case was still an open problem. For the first time we prove in this paper that this scheme is secure under the assumptions that the hash function closely behaves like a random oracle and that the computational Diffie-Hellman problem is difficult. Furthermore, since Denial-of-Service (DoS) attacks have become a common threat we enhance AuthA with a mechanism to protect against them.

1 Introduction

The need for authentication is obvious when two entities communicate on the Internet. However, proving knowledge of a secret over a public link without leaking any information about this secret is a complex process. One extreme example is when a short string is used by a human as a means to get access to a remote service. This password is used by the human to authenticate itself to the remote service in order to establish a session key to be used to implement an authenticated communication channel within which messages set over the wire are cryptographically protected. Humans directly benefit from this approach since they only need to remember a low-quality string chosen from a relatively small dictionary (i.e. 4 decimal digits).

The seminal work in this area is the *Encrypted Key Exchange* (EKE) protocol proposed by Bellare and Merritt in [5, 6]. EKE is a classical Diffie-Hellman key exchange wherein the two flows are encrypted using the password as a common symmetric key. This encryption primitive can be instantiated via either a password-keyed symmetric cipher or a mask generation function computed as the product of the message with a hash of the password. This efficient structure later evolved into a protocol named AuthA considered for standardization by the IEEE P1363 Standard working group on public-key cryptography [3]. AuthA was conjectured secure against dictionary attacks by its designers, but actually proving it was left as an open problem.

Cryptographers have begun to analyze the AuthA protocol in an ideal model of computation wherein a hash function is modeled via a random function and a block cipher is modeled via random permutations [2, 5, 8]. These analyses have provided useful arguments in favor of AuthA, but do not guarantee that AuthA is secure in the real world. These analyses only show that AuthA is secure against generic attacks that do not exploit a particular implementation of the block cipher, but in practice current block ciphers are far from being random permutations. A security proof in the random-oracle model only, while still using ideal objects, would provide a stronger and more convincing argument in favor of AuthA.

One should indeed note that the ideal-cipher model seems to be a stronger model than the random-oracle one. Even if one knows constructions to build random permutations from random functions [12], they cannot be used to build ideal ciphers from random oracles. The difference here comes from the fact that the inner functions (random oracles) are available to the adversary. It could compute plaintext-ciphertext relations starting from the middle of the Feistel network, while in the *programmable* ideal-cipher model, one needs to control all these relations.

Moreover, a AuthA scheme resistant to Denial-of-Service (DoS) attacks would be more suited to the computing environment we face every day since nowadays through the Internet hackers make servers incapable of accepting new connections. These so-called *Distributed DoS attacks* exhaust the memory and computational power of the servers.

Contributions. This paper examines the security of the AuthA password-authenticated key exchange protocol in the random-oracle model under the computational Diffie-Hellman assumption; no ideal-cipher assumption is needed. We work out our proofs by first defining the execution of AuthA in the communication model of Bellare *et al.* [2] and then adapting the proof techniques recently published by Bresson *et al.* [8]. We exhibit very compact and “elegant” proofs to show that the *One-Mask* (OMDHKE— one flow is encrypted only) and the *Two-Mask* (MDHKE— both flows are encrypted) formal variants of AuthA and EKE are secure in the random-oracle model when the encryption primitive is a mask generation function.

We define the execution of AuthA in the Bellare *et al.*’s model wherein the protocol entities are modeled through oracles, and the various types of attacks are modeled by queries to these oracles. This model enables a treatment of dictionary attacks by allowing the adversary to obtain honest executions of the AuthA protocol. The security of AuthA against dictionary attacks depends on how many interactions the adversary carries out against the protocol entities rather than on the adversary’s computational power.

We furthermore enhance the schemes with a mechanism that offers protection against Denial-of-Service (DoS) attacks. This mechanism postpones the computation of any exponentiations on the server side, as well as the storage of any states, after that the initiator of the connection has been identified as being a legitimate client. Roughly speaking, the server sends to the client a “puzzle” [11] to solve which will require from the client to perform multiple cryptographic computations while the server can easily and efficiently check that the solution is correct.

Related Work. The IEEE P1363.2 Standard working group on password-based authenticated key-exchange methods [10] has been focusing on key exchange protocols wherein clients use short passwords in place of certificates to identify themselves to servers. This standardization effort has its roots in the works of Bellare *et al.* [2] and Boyko *et al.* [7], wherein formal models and security goals for password-based key agreement were first formulated. Bellare *et al.* analyzed the EKE (where EKE stands for *Encrypted Key Exchange*) protocol [5], a classical Diffie-Hellman key exchange wherein the two flows are encrypted using the password as a common symmetric key. Several proofs have already been proposed, in various models, but all very intricate. The present paper provides a very short and “elegant” proof of AuthA or OMDHKE (but also of EKE or MDHKE in the appendix), that is less prone to errors.

Several works have already focused on designing mechanisms to protect against DoS attacks. Aiello *et al.* [1] treat the amount of Perfect Forward-Secrecy (PFS) as an engineering parameter that can be traded off against resistance to DoS attacks. DoS-resistance is achieved by saving the “state” of the current session in the protocol itself (i.e., in the flows) rather than on the server side. More precisely, the “state” of the protocol is hashed and put into a *cookie*, while the server needs only to memorize the hash value. Only once this is done, the server saves the full

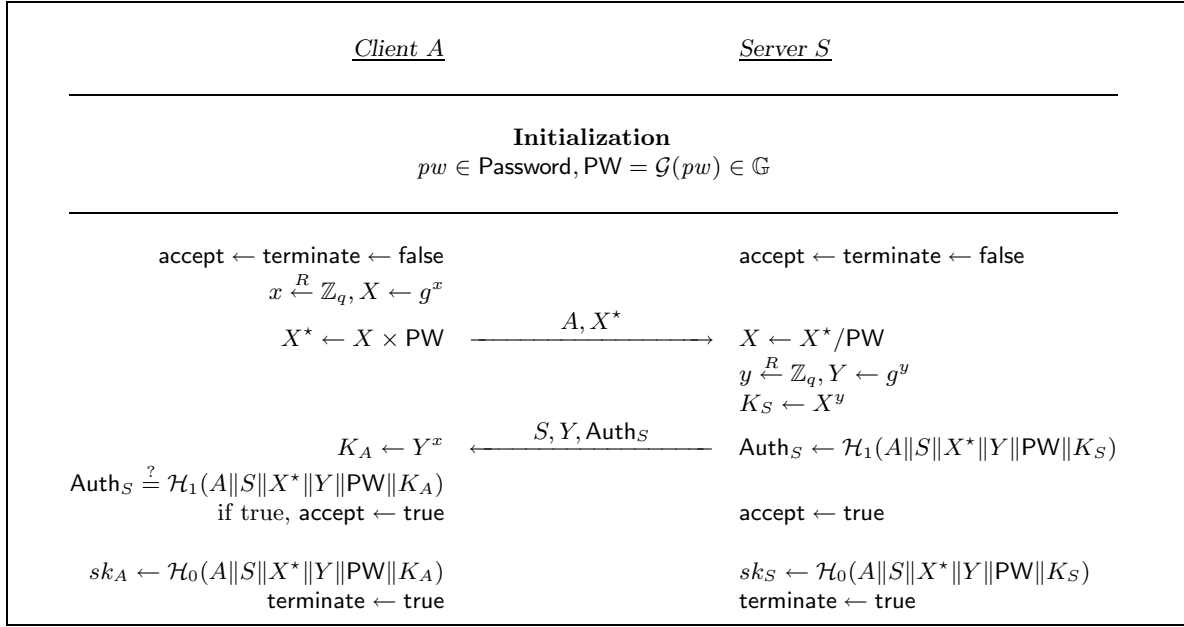


Fig. 1. An execution of the protocol OMDHKE, run between a client and a server.

state and the connection is established. This technique prevents the attacker from exhausting the server's memory but do not prevent it from exhausting the server's computational power. One approach to counter the latter threat is to make the client compute some form of proof of computational effort, using a “puzzle” [11], also more recently used by Dwork *et al.* [9] to discourage spam. The present paper builds on that latter concept.

2 The OMDHKE Protocol: One-Mask Diffie-Hellman Key Exchange

The arithmetic is in a finite cyclic group $\mathbb{G} = \langle g \rangle$ of order a ℓ -bit prime number q , where the operation is denoted multiplicatively. We also denote by $\mathbb{G}^*$ the subset $\mathbb{G} \setminus \{1\}$ of the generators of $\mathbb{G}$. Hash functions from $\{0, 1\}^*$ to $\{0, 1\}^{\ell_i}$ are denoted $\mathcal{H}_i$, for $i = 0, 1$. While $\mathcal{G}$ denotes a full-domain hash function from $\{0, 1\}^*$ into $\mathbb{G}$. As illustrated on Figure 1 (with an honest execution of the OMDHKE protocol), the protocol runs between two parties A and S , and the session-key space $\mathbf{SK}$ associated to this protocol is $\{0, 1\}^{\ell_0}$ equipped with a uniform distribution.

The parties initially share a low-quality string pw , the password, drawn from the dictionary Password according to the distribution $\mathcal{D}_{pw}$. In the following, we use the notation $\mathcal{D}_{pw}(q)$ for the probability to be in the most probable set of q passwords:

$$\mathcal{D}_{pw}(q) = \max_{P \subseteq \text{Password}} \left\{ \Pr_{pw \in \mathcal{D}_{pw}} [pw \in P \mid \#P \leq q] \right\}.$$

Note that if we denote by $\mathcal{U}_N$ the uniform distribution among N passwords, $\mathcal{U}_N(q) = q/N$.

The protocol then runs as follows. The client chooses at random a private random exponent x and computes the corresponding Diffie-Hellman public value g^x , but does not send this last value in the clear. The client encrypts the Diffie-Hellman public value using a mask generation function as the product of a Diffie-Hellman value with a full-domain hash of the password. Upon receiving this encrypted value, the server unmaskes it and computes the Diffie-Hellman secret value g^{xy} which is used by the server to compute its authenticator Auth_S and the session key. The

server sends its Diffie-Hellman public value g^y in the clear, Auth_S , and terminates the execution of the protocol. Upon receiving these values, the client computes the secret Diffie-Hellman value and checks that the authenticator Auth_S is a valid one. If the authenticator is valid, the client computes the session key, and terminates the execution of the protocol.

3 The Formal Model

The security model is the same as the one defined by Bellare *et al.* [2]. We briefly review it.

The Security Model. We denote by A and S two parties that can participate in the key exchange protocol P . Each of them may have several *instances* called oracles involved in distinct, possibly concurrent, executions of P . We denote A (resp. S) instances by A^i (resp. S^j), or by U when we consider any user instance. The two parties share a low-entropy secret pw which is drawn from a small dictionary Password , according to the distribution $\mathcal{D}_{pw}$.

The key exchange algorithm P is an interactive protocol between A^i and S^j that provides the instances of A and S with a session key sk . During the execution of this protocol, the adversary has the entire control of the network, and tries to break the privacy of the key, or the authentication of the players. To this aim, several queries are available to it. Let us briefly recall the capability that each query captures:

- $\text{Execute}(A^i, S^j)$: This query models passive attacks, where the adversary gets access to honest executions of P between the instances A^i and S^j by eavesdropping.
- $\text{Reveal}(U)$: This query models the misuse of the session key by instance U (*known-key attacks*). The query is only available to $\mathcal{A}$ if the attacked instance actually “holds” a session key and it releases the latter to $\mathcal{A}$.
- $\text{Send}(U, m)$: This query enables to consider active attacks by having $\mathcal{A}$ sending a message to instance U . The adversary $\mathcal{A}$ gets back the response U generates in processing the message m according to the protocol P . A query $\text{Send}(A^i, \text{Start})$ initializes the key exchange algorithm, and thus the adversary receives the initial flow the player A should send out to the player S .

In the active scenario, the Execute -query may at first seem useless since using the Send -query the adversary has the ability to carry out honest executions of P among parties. Yet, even in this scenario, the Execute -query is essential for properly dealing with dictionary attacks. The number q_s of Send -queries directly asked by the adversary does not take into account the number of Execute -queries. Therefore, q_s represents the number of flows the adversary has built by itself, and therefore the number of passwords it would have tried.

Security Notions. As already noticed, the aim of the adversary is to break the privacy of the session key (a.k.a., semantic security) or the authentication of the players (having a player accepting while no instance facing him). The security notions take place in the context of executing P in the presence of the adversary $\mathcal{A}$. One first draws a password pw from Password according to the distribution $\mathcal{D}_{pw}$, provides coin tosses to $\mathcal{A}$, all oracles, and then runs the adversary by letting it ask any number of queries as described above, in any order.

AKE Security. The privacy (semantic security) of the session key is modeled by the game $\text{Game}^{\text{ake}}(\mathcal{A}, P)$, in which one more query is available to the adversary: $\text{Test}(U)$. The Test -query can be asked at most once by the adversary $\mathcal{A}$ and is only available to $\mathcal{A}$ if the attacked

instance U is **Fresh** (which roughly means that the session key is not “obviously” known to the adversary.) This query is answered as follows: one flips a (private) coin b and forwards sk (the value $\text{Reveal}(U)$ would output) if $b = 1$, or a random value if $b = 0$. When playing this game, the goal of the adversary is to guess the bit b involved in the **Test**-query, by outputting this guess b' . We denote the **AKE advantage** as the probability that $\mathcal{A}$ correctly guesses the value of b . More precisely we define $\text{Adv}_P^{\text{ake}}(\mathcal{A}) = 2\Pr[b = b'] - 1$. The protocol P is said to be (t, ε) -**AKE-secure** if $\mathcal{A}$'s advantage is smaller than ε for any adversary $\mathcal{A}$ running with time t .

Authentication. Another goal is to consider *unilateral authentication* of either A (A -Auth) or S (S -Auth) wherein the adversary impersonates a party. We denote by $\text{Succ}_P^{A\text{-auth}}(\mathcal{A})$ (resp. $\text{Succ}_P^{S\text{-auth}}(\mathcal{A})$) the probability that $\mathcal{A}$ successfully impersonates an A instance (resp. an S instance) in an execution of P , which means that S (resp. A) agrees on a key, while the latter is shared with no instance of A (resp. S). A protocol P is said to be (t, ε) -**Auth-secure** if $\mathcal{A}$'s success for breaking either A -Auth or S -Auth is smaller than ε for any adversary $\mathcal{A}$ running with time t .

3.1 Computational Diffie-Hellman Assumption

A (t, ε) -CDH $_{g, \mathbb{G}}$ attacker, in a finite cyclic group $\mathbb{G}$ of prime order q with g as a generator, is a probabilistic machine Δ running in time t such that its success probability $\text{Succ}_{g, \mathbb{G}}^{\text{cdh}}(\Delta)$, given random elements g^x and g^y to output g^{xy} , is greater than ε . As usual, we denote by $\text{Succ}_{g, \mathbb{G}}^{\text{cdh}}(t)$ the maximal success probability over every adversaries running within time t . The CDH-Assumption states that $\text{Succ}_{g, \mathbb{G}}^{\text{cdh}}(t) \leq \varepsilon$ for any t/ε not too large.

4 Security Proof for the OMDHKE Protocol

In this section we show that the OMDHKE protocol distributes session keys that are semantically-secure and provides *unilateral authentication* of the server S . The specification of this protocol is found on Figure 1.

Theorem 1 (AKE/UA Security). *Let us consider the protocol OMDHKE, over a group of prime order q , where Password is a dictionary equipped with the distribution $\mathcal{D}_{pw}$. For any adversary $\mathcal{A}$ within a time bound t , with less than q_s active interactions with the parties (Send-queries) and q_p passive eavesdroppings (Execute-queries), and asking q_g and q_h hash queries to $\mathcal{G}$ and any $\mathcal{H}_i$ respectively,*

$$\begin{aligned} \text{Adv}_{\text{omdhke}}^{\text{ake}}(\mathcal{A}) &\leq \frac{2q_s}{2^{\ell_1}} + 12 \times \mathcal{D}_{pw}(q_s) + 12q_h^2 \times \text{Succ}_{g, \mathbb{G}}^{\text{cdh}}(t + 2\tau_e) + \frac{2Q^2}{q}, \\ \text{Succ}_{\text{omdhke}}^{S\text{-auth}}(\mathcal{A}) &\leq \frac{q_s}{2^{\ell_1}} + 3 \times \mathcal{D}_{pw}(q_s) + 3q_h^2 \times \text{Succ}_{g, \mathbb{G}}^{\text{cdh}}(t + 3\tau_e) + \frac{Q^2}{2q}, \end{aligned}$$

where $Q = q_p + q_s + q_g$ and τ_e denotes the computational time for an exponentiation in $\mathbb{G}$.

This theorem shows that the protocol is secure against dictionary attacks since the advantage of the adversary essentially grows with the ratio of interactions (number of **Send**-queries) to the number of passwords.

Proof. In this proof, we incrementally define a sequence of games starting at the real game $\mathbf{G}_0$ and ending up at $\mathbf{G}_5$. We use the Shoup's lemma [13] to bound the probability of each event in these games.

Game $\mathbf{G}_0$: This is the real protocol, in the random-oracle model. We are interested in the two following events:

- S_0 (for semantic security), which occurs if the adversary correctly guesses the bit b involved in the **Test**-query;
- A_0 (for S -authentication), which occurs if an instance A^i accepts with no partner instance S^j (with the same transcript $((A, X^*), (S, Y, \text{Auth}))$).

$$\text{Adv}_{\text{omdhke}}^{\text{ake}}(\mathcal{A}) = 2 \Pr[S_0] - 1 \quad \text{Succ}_{\text{omdhke}}^{S\text{-auth}}(\mathcal{A}) = \Pr[A_0]. \quad (1)$$

Actually, in any game $\mathbf{G}_n$ below, we study the event A_n , and the restricted event $\text{SA}_n = S_n \wedge \neg A_n$.
Game $\mathbf{G}_1$: In this game, we simulate the hash oracles $(\mathcal{G}, \mathcal{H}_0 \text{ and } \mathcal{H}_1)$, but also additional hash functions, for $i = 0, 1$: $\mathcal{H}'_i : \{0, 1\}^* \rightarrow \{0, 1\}^{\ell_i}$ that will appear in the Game $\mathbf{G}_3$) as usual by maintaining hash lists $\Lambda_{\mathcal{G}}$, $\Lambda_{\mathcal{H}}$ and $\Lambda_{\mathcal{H}'}$ (see Figure 2). We also simulate all the instances, as the real players would do, for the **Send**-queries and for the **Execute**, **Reveal** and **Test**-queries (see Figure 3). From this simulation, we easily see that the game is perfectly indistinguishable from the real attack.

$\mathcal{G}$ and $\mathcal{H}_i$ oracles	For a hash-query $\mathcal{H}_i(q)$ (resp. $\mathcal{H}'_i(q)$), such that a record (i, q, r) appears in $\Lambda_{\mathcal{H}}$ (resp. $\Lambda_{\mathcal{H}'}$), the answer is r . Otherwise one chooses a random element $r \in \{0, 1\}^{\ell_i}$, answers with it, and adds the record (i, q, r) to $\Lambda_{\mathcal{H}}$ (resp. $\Lambda_{\mathcal{H}'}$).
	For a hash-query $\mathcal{G}(q)$ such that a record $(q, r, \star)$ appears in $\Lambda_{\mathcal{G}}$, the answer is r . Otherwise the answer r is defined according to the following rule:
	► Rule $\mathcal{G}^{(1)}$ Choose a random element $r \in \mathbb{G}$. The record $(q, r, \perp)$ is added to $\Lambda_{\mathcal{G}}$. Note: the third component of the elements of this list will be explained later.

Fig. 2. Simulation of the hash functions

Game $\mathbf{G}_2$: For an easier analysis in the following, we cancel games in which some (unlikely) collisions appear:

- collisions on the partial transcripts $((A, X^*), (S, Y))$. Note that transcripts involve at least one honest party, and thus one of X^* or Y is truly uniformly distributed;
- collisions on the output of $\mathcal{G}$.

Both probabilities are bounded by the birthday paradox:

$$\Pr[\text{Coll}_2] \leq \frac{(q_p + q_s)^2}{2q} + \frac{q_g^2}{2q}. \quad (2)$$

Game $\mathbf{G}_3$: We compute the session key sk and the authenticator Auth using the private oracles $\mathcal{H}'_0$ and $\mathcal{H}'_1$ respectively:

► **Rule A3/S3⁽³⁾**

- | Compute the authenticator $\text{Auth} = \mathcal{H}'_1(A \| S \| X^* \| Y)$.
- | Compute the session key $sk_{A/S} = \mathcal{H}'_0(A \| S \| X^* \| Y)$.

Since we do no longer need to compute the values K_A and K_S , we can simplify the second rules:

► **Rule A2/S2⁽³⁾**

- | Do nothing.

Send-queries to A	We answer to the Send-queries to an A-instance as follows: – A Send(A^i, Start)-query is processed according to the following rule: ► Rule A1⁽¹⁾ - Choose a random exponent $\theta \in \mathbb{Z}_q$, compute $X = g^\theta$ and $X^* = X \times \text{PW}$. Then the query is answered with (A, X^*), and the instance goes to an expecting state. – If the instance A^i is in an expecting state, a query Send($A^i, (S, Y, \text{Auth})$) is processed by computing the authenticator and the session key. We apply the following rules: ► Rule A2⁽¹⁾ - Compute $K_A = Y^\theta$. ► Rule A3⁽¹⁾ - Compute the authenticator and the session key: $\text{Auth}' = \mathcal{H}_1(A \ S \ X^* \ Y \ \text{PW} \ K_A);$ $sk_A = \mathcal{H}_0(A \ S \ X^* \ Y \ \text{PW} \ K_A).$ If $\text{Auth} = \text{Auth}'$, the instance accepts. In any case, the instance terminates.
Send-queries to S	We answer to the Send-queries to a S-instance as follows: – A Send($S^j, (A, X^*)$)-query is processed according to the following rules: ► Rule S1⁽¹⁾ - Choose a random exponent $\varphi \in \mathbb{Z}_q$, compute $Y = g^\varphi$. Then, the instance compute the authenticator and session key. We apply the following rules: ► Rule S2⁽¹⁾ - Compute $X = X^* / \text{PW}$ and $K_S = X^\varphi$. ► Rule S3⁽¹⁾ - Compute the authenticator and the session key: $\text{Auth} = \mathcal{H}_1(A \ S \ X^* \ Y \ \text{PW} \ K_S);$ $sk_S = \mathcal{H}_0(A \ S \ X^* \ Y \ \text{PW} \ K_S).$ Then the query is answered with (S, Y, Auth), and the instance accepts and terminates.
Other queries	An Execute(A^i, S^j)-query is processed using successively the above simulations of the Send-queries: $(A, X^*) \leftarrow \text{Send}(A^i, \text{Start})$ and $(S, Y, \text{Auth}) \leftarrow \text{Send}(S^j, (A, X^*))$, and then outputting the transcript $((A, X^*), (S, Y, \text{Auth}))$. A Reveal($U$)-query returns the session key (sk_A or sk_S) computed by the instance U (if the latter has accepted). A Test(U)-query first gets sk from Reveal(U), and flips a coin b. If $b = 1$, we return the value of the session key sk, otherwise we return a random value drawn from $\{0, 1\}^\ell$.

Fig. 3. Simulation of the OMDHKE protocol

Finally, one can note that the password is not used anymore either, then we can also simplify the generation of X^* , using the group property of $\mathbb{G}$:

► **Rule A1**⁽³⁾

- | Choose a random element $x \in \mathbb{Z}_q$ and compute $X^* = g^x$.

The games $\mathbf{G}_3$ and $\mathbf{G}_2$ are indistinguishable unless some specific hash queries are asked, denoted by event $\text{AskH}_3 = \text{AskH0w1}_3 \vee \text{AskH1}_3$:

- AskH1_3 : $\mathcal{A}$ queries $\mathcal{H}_1(A \| S \| X^* \| Y \| \text{PW} \| K_A)$ or $\mathcal{H}_1(A \| S \| X^* \| Y \| \text{PW} \| K_S)$ for some execution transcript $((A, X^*), (S, Y, \text{Auth}))$;
- AskH0w1_3 : $\mathcal{A}$ queries $\mathcal{H}_0(A \| S \| X^* \| Y \| \text{PW} \| K_A)$ or $\mathcal{H}_0(A \| S \| X^* \| Y \| \text{PW} \| K_S)$ for some execution transcript $((A, X^*), (S, Y, \text{Auth}))$, where some party has accepted, but event AskH1_3 did not happen.

The authenticator is computed with a random oracle that is private to the simulator, then one can remark that it cannot be guessed by the adversary, better than at random for each attempt, unless the same partial transcript $((A, X^*), (S, Y))$ appeared in another session with a real instance S^j . But such a case has already been excluded (in Game $\mathbf{G}_2$). A similar remark can be led about the session key:

$$\Pr[A_3] \leq \frac{q_s}{2^{\ell_1}} \quad \Pr[SA_3] = \frac{1}{2}. \quad (3)$$

When collisions of partial transcripts have been excluded, the event **AskH1** can be split in 3 disjoint sub-cases:

- **AskH1-Passive₃**: the transcript $((A, X^*), (S, Y, \text{Auth}))$ comes from an execution between instances of A and S (Execute-queries or forward of **Send**-queries, replay of part of them). This means that both X^* and Y have been simulated;
- **AskH1-WithA₃**: the execution involved an instance of A , but Y has not been sent by any instance of S . This means that X^* has been simulated, but Y has been produced by the adversary;
- **AskH1-WithS₃**: the execution involved an instance of S , but X^* has not been sent by any instance of A . This means that Y has been simulated, but X^* has been produced by the adversary.

Game $\mathbf{G}_4$: In order to evaluate the above events, we introduce a random Diffie-Hellman instance (P, Q) , (with both $P \in \mathbb{G}^*$ and $Q \in \mathbb{G}^*$, which are thus generators of $\mathbb{G}$. Otherwise, the Diffie-Hellman problem is easy.) We first modify the simulation of the oracle $\mathcal{G}$, involving the element Q . The simulation introduces values in the third component of the elements of $\Lambda_{\mathcal{G}}$, but does not use it.

► **Rule $\mathcal{G}^{(4)}$**

| Choose a random element $k \in \mathbb{Z}_q^*$ and compute $r = Q^{-k}$. The record (q, r, k) is added to $\Lambda_{\mathcal{G}}$.

We introduce the other part P of the Diffie-Hellman instance in the simulation of the party S .

► **Rule $\mathbf{S1}^{(4)}$**

| Choose a random element $y \in \mathbb{Z}_q^*$ and compute $Y = P^y$.

It would let the probabilities unchanged, but note that we excluded the cases $\text{PW} = 1$ and $Y = 1$:

$$|\Pr[\text{AskH}_4] - \Pr[\text{AskH}_3]| \leq \frac{q_s + q_p}{q} + \frac{q_g}{q}. \quad (4)$$

Game $\mathbf{G}_5$: It is now possible to evaluate the probability of the event **AskH** (or more precisely, the sub-cases). Indeed, one can remark that the password is never used during the simulation, it can be chosen at the very end only. Then, an information-theoretic analysis can be performed, which simply uses cardinalities of some sets.

To this aim, we first cancel a few more games, wherein for some pairs $(X^*, Y) \in \mathbb{G}^2$, involved in a communication between **an instance S^j** and either the adversary or an instance A^i , there are two distinct elements **PW** such that the tuple $(X^*, Y, \text{PW}, \text{CDH}_{g, \mathbb{G}}(X^*/\text{PW}, Y))$ is in $\Lambda_{\mathcal{H}}$ (which event is denoted **CollH₅**):

$$|\Pr[\text{AskH}_5] - \Pr[\text{AskH}_4]| \leq \Pr[\text{CollH}_5]. \quad (5)$$

Hopefully, event **CollH₅** can be upper-bounded, granted the following Lemma:

Lemma 2. *If for some pair $(X^*, Y) \in \mathbb{G}^2$, involved in a communication with an instance S^j , there are two elements PW_0 and PW_1 such that (X^*, Y, PW_i, Z_i) are in $\Lambda_{\mathcal{H}}$ with $Z_i = \text{CDH}_{g, \mathbb{G}}(X^*/PW_i, Y)$, one can solve the computational Diffie-Hellman problem:*

$$\Pr[\text{CollH}_5] \leq q_h^2 \times \text{Succ}_{g, \mathbb{G}}^{\text{cdh}}(t + \tau_e). \quad (6)$$

Proof. Assume there exist such elements $(X^*, Y = P^y) \in \mathbb{G}^2$, $PW_0 = Q^{-k_0}$, and $PW_1 = Q^{-k_1}$. Note that

$$\begin{aligned} Z_i &= \text{CDH}_{g, \mathbb{G}}(X^*/PW_i, Y) = \text{CDH}_{g, \mathbb{G}}(X^* \times Q^{k_i}, Y) \\ &= \text{CDH}_{g, \mathbb{G}}(X^*, Y) \times \text{CDH}_{g, \mathbb{G}}(Q, Y)^{k_i} = \text{CDH}_{g, \mathbb{G}}(X^*, Y) \times \text{CDH}_{g, \mathbb{G}}(P, Q)^{y k_i}. \end{aligned}$$

As a consequence, $Z_1/Z_0 = \text{CDH}_{g, \mathbb{G}}(P, Q)^{y(k_1 - k_0)}$, and thus $\text{CDH}_{g, \mathbb{G}}(P, Q) = (Z_1/Z_0)^u$, where u is the inverse of $y(k_1 - k_0)$ in $\mathbb{Z}_q$. The latter exists since $PW_1 \neq PW_2$, and $y \neq 0$. By guessing the two queries asked to the $\mathcal{H}_i$, one concludes the proof. $\square$

In order to conclude, let us study separately the three sub-cases of AskH1 and then AskH0w1 (keeping in mind the absence of several kinds of collisions: for partial transcripts, for $\mathcal{G}$, and for PW in $\mathcal{H}$ -queries):

- AskH1-Passive: About the passive transcripts (in which both X^* and Y have been simulated), one can state the following lemma:

Lemma 3. *If for some pair $(X^*, Y) \in \mathbb{G}^2$, involved in a passive transcript, there is an element PW such that (X^*, Y, PW, Z) is in $\Lambda_{\mathcal{H}}$, with $Z = \text{CDH}_{g, \mathbb{G}}(X^*/PW, Y)$, one can solve the computational Diffie-Hellman problem:*

$$\Pr[\text{AskH1-Passive}_5] \leq q_h \times \text{Succ}_{g, \mathbb{G}}^{\text{cdh}}(t + 2\tau_e).$$

Proof. Assume there exist such elements $(X^* = g^x, Y = P^y) \in \mathbb{G}^2$ and $PW = Q^{-k}$. As above,

$$Z = \text{CDH}_{g, \mathbb{G}}(X^*, Y) \times \text{CDH}_{g, \mathbb{G}}(Q, Y)^k = P^{xy} \times \text{CDH}_{g, \mathbb{G}}(P, Q)^{yk}.$$

As a consequence, $\text{CDH}_{g, \mathbb{G}}(P, Q) = (Z/P^{xy})^u$, where u is the inverse of yk in $\mathbb{Z}_q$. The latter exists since we have excluded the cases where $y = 0$ or $k = 0$. By guessing the query asked to the $\mathcal{H}_i$, one concludes the proof. $\square$

- AskH1-WithA: this event may correspond to an attack where the adversary tries to impersonate S to A (break unilateral authentication). But each authenticator sent by the adversary has been computed with at most one PW value. Without any $\mathcal{G}$ -collision, it corresponds to at most one pw :

$$\Pr[\text{AskH1-WithA}_5] \leq \mathcal{D}_{pw}(q_s).$$

- AskH1-WithS: The above Lemma 2, when applied to games where the event CollH₅ did not happen (and without $\mathcal{G}$ -collision), states that for each pair (X^*, Y) involved in a transcript with an instance S^j , there is at most one element pw such that for $PW = \mathcal{G}(pw)$ the corresponding tuple is in $\Lambda_{\mathcal{H}}$: the probability over a random password is thus less than $\mathcal{D}_{pw}(q_s)$. As a consequence,

$$\Pr[\text{AskH1-WithS}_5] \leq \mathcal{D}_{pw}(q_s).$$

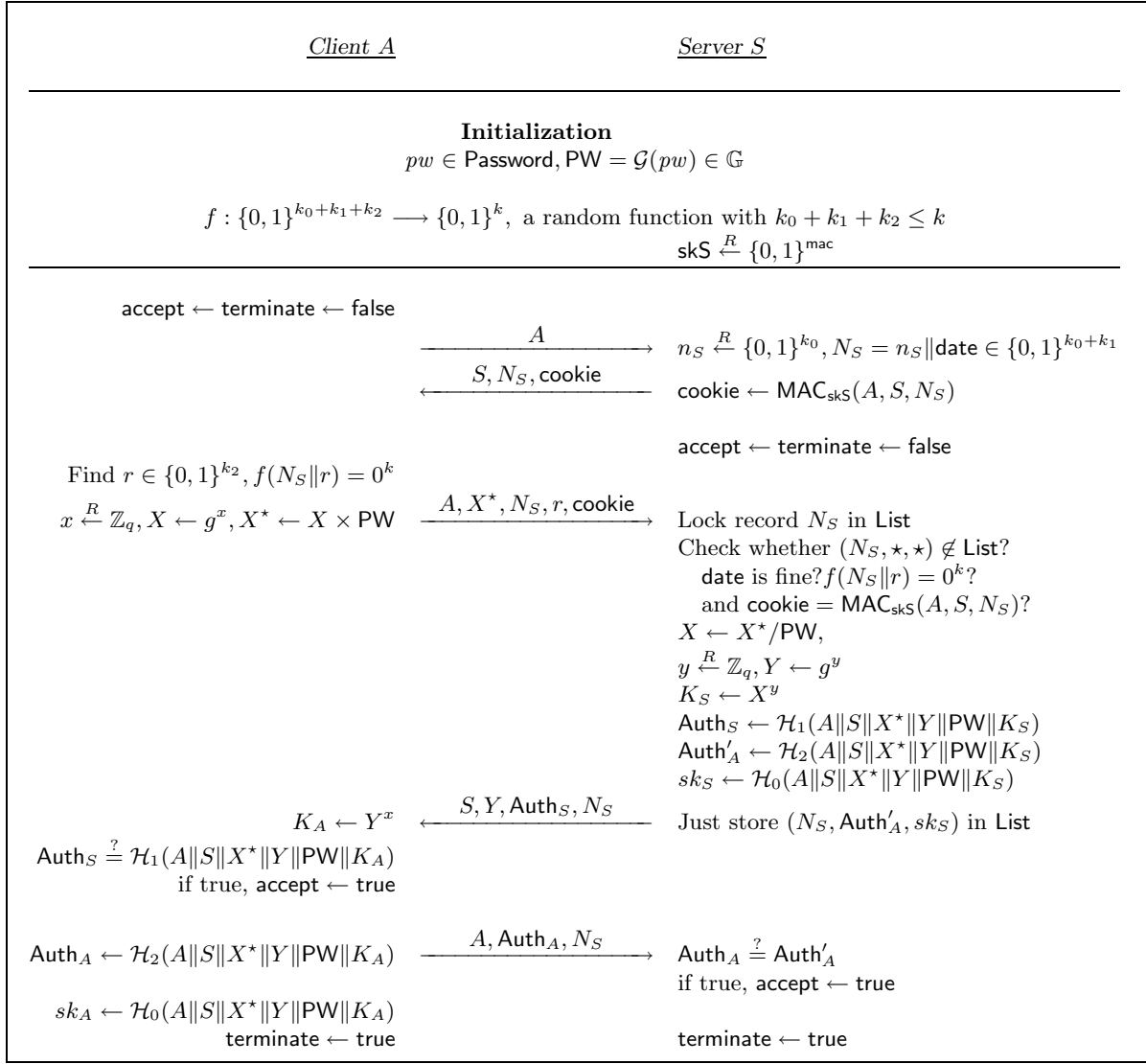


Fig. 4. An execution of the protocol OMDHKE, run between a client and a server, enhanced with mutual authentication and a denial-of-service protection.

About AskH0w1 (when the three above events did not happen), it means that only executions with an instance of S (and either A or the adversary) may lead to acceptance. Exactly the same analysis as for AskH1-Passive and AskH1-WithS leads to $\Pr[\text{AskH0w1}_5] \leq \mathcal{D}_{pw}(q_s) + q_h \times \text{Succ}_{g, \mathbb{G}}^{\text{cdh}}(t + 2\tau_e)$. As a conclusion,

$$\Pr[\text{AskH}_5] \leq 3\mathcal{D}_{pw}(q_s) + 2q_h \times \text{Succ}_{g, \mathbb{G}}^{\text{cdh}}(t + 2\tau_e). \quad (7)$$

Combining all the above equations, one gets the announced result. $\square$

5 The DoS-Resistant OMDHKE Protocol

In a computing environment where Distributed DoS attacks are a continual threat, a server needs to protect itself from non-legitimate clients that will exhaust its memory and computational power. Intensive cryptographic computations (i.e. exponentiation), as well as states, are only

performed after a client proves to the server that it was able to solve a given “puzzle”. The “puzzle” is chosen so that the client can only solve it by exhaustive search while the server can quickly checks whether a given proposition solves it. This “puzzle” is chosen as follows.

The server first picks at random a MAC-symmetric key that it will use to authenticate *cookie*; the MAC-key is used across multiple connections. The server then forms the authenticated *cookie* which is the MAC of a random nonce and the date, and sends it to the client. The precision of the date is determined according to the level of DoS required. The use of a cookie makes the protocol stateless on the server side. Upon receiving the cookie, the client tries to find an input which hashes to the NULL value. Since this hash function is seen as a random oracle, the only way for the client to solve this “puzzle” is to run through all possible prefixed strings and query the random oracle [4]. Later in practice this function is instantiated using specific functions derived from standard hash functions such as SHA1. Once the client has found such a proof of computational effort, it sends it back with the authenticated cookie and its Diffie-Hellman public value to the server. Upon receiving these values the server checks whether the client is launching a DoS attack by initiating several connections in parallel and replaying this proof of computational effort on another connection. The server reaches this aim by locking the cookie and not admitting the same cookie twice (hence the date in this challenge is used to tune the size of the database). If all the checks verify, the server starts saving states and computing the necessary exponentiations to establish a session key. From this point on the protocol works as the original AuthA protocol, adding mutual authentication [2].

6 Conclusion

The above proof does not deal with forward-secrecy. Forward-secrecy entails that the corruption of the password does not compromise the semantic security of previously established session keys. One could easily prove that this scheme achieves forward secrecy, as in [8], while losing a quadratic factor in the reduction.

In conclusion, this paper provides strong security arguments that support the standardization of the AuthA protocol by the IEEE P1363.2 Standard working group on password-based public key cryptography . We have presented a compact and “elegant” proof of security for the AuthA protocol [3] when the symmetric-encryption primitive is instantiated using a mask generation function, which extends our previous work when the symmetric-encryption primitive is assumed to behave like an ideal cipher [8]. The security of the protocol was indeed stated as an open problem by its designers. In our study, the symmetric encryption basic block takes the form of a multiplication in the Diffie-Hellman group. Our result is a significant departure from previous known results since the security of AuthA can now be based on weaker and more reasonable assumptions involving both the random-oracle model and the computational Diffie-Hellman problem. Moreover, we investigate and propose a practical, reasonable solution to make the protocol secure against DoS attacks. One can also find further studies on the variant in which both flows are encrypted between the client and the server in the appendix.

Acknowledgments

The second author was supported by the Director, Office of Science, Office of Advanced Scientific Computing Research, Mathematical Information and Computing Sciences Division, of the U.S. Department of Energy under Contract No. DE-AC03-76SF00098. This document is report LBNL-53099. Disclaimer available at <http://www-library.lbl.gov/disclaimer>.

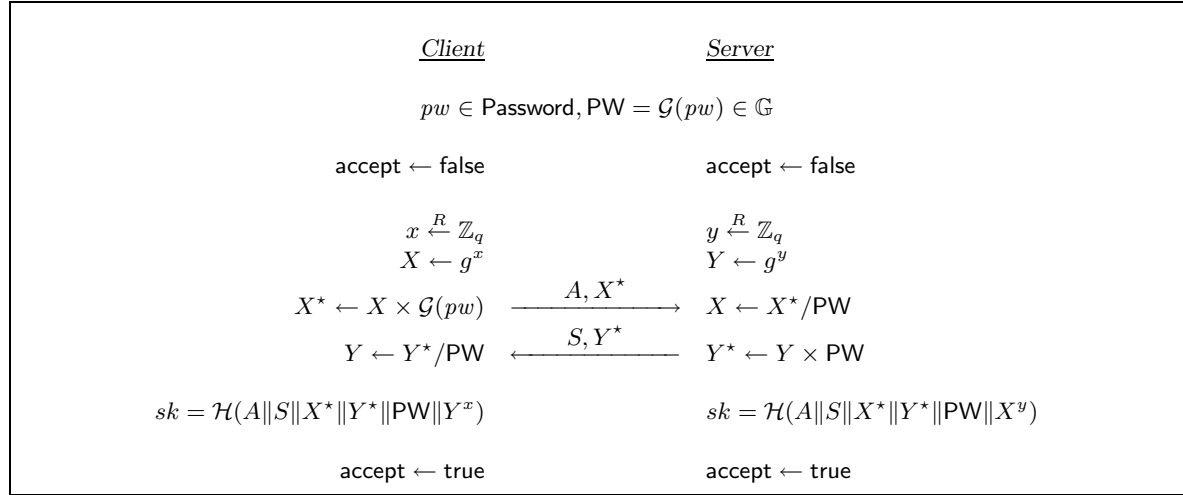


Fig. 5. An execution of the protocol MDHKE.

References

1. W. Aiello, S. M. Bellare, M. Blaze, R. Canetti, J. Ioannidis, A. D. Keromytis, and O. Reingold. Efficient, DoS-resistant, Secure Key Exchange for Internet Protocols. In *Proc. of the 9th CCS*, pages 48–58. ACM Press, New York, 2002.
2. M. Bellare, D. Pointcheval, and P. Rogaway. Authenticated Key Exchange Secure Against Dictionary Attacks. In *Eurocrypt '00*, LNCS 1807, pages 139–155. Springer-Verlag, Berlin, 2000.
3. M. Bellare and P. Rogaway. The AuthA Protocol for Password-Based Authenticated Key Exchange. Contributions to IEEE P1363. March 2000.
Available from <http://grouper.ieee.org/groups/1363/>.
4. M. Bellare and P. Rogaway. Random Oracles Are Practical: a Paradigm for Designing Efficient Protocols. In *Proc. of the 1st CCS*, pages 62–73. ACM Press, New York, 1993.
5. S. M. Bellare and M. Merritt. Encrypted Key Exchange: Password-Based Protocols Secure against Dictionary Attacks. In *Proc. of the Symposium on Security and Privacy*, pages 72–84. IEEE, 1992.
6. S. M. Bellare and M. Merritt. Augmented Encrypted Key Exchange: A Password-Based Protocol Secure against Dictionary Attacks and Password File Compromise. In *Proc. of the 1st CCS*, pages 244–250. ACM Press, New York, 1993.
7. V. Boyko, P. MacKenzie, and S. Patel. Provably Secure Password Authenticated Key Exchange Using Diffie-Hellman. In *Eurocrypt '00*, LNCS 1807, pages 156–171. Springer-Verlag, Berlin, 2000.
Final version available at: <http://cm.bell-labs.com/who/philmac/research/>.
8. E. Bresson, O. Chevassut, and D. Pointcheval. Security Proofs for Efficient Password-Based Key Exchange. In *Proc. of the 10th CCS*. ACM Press, New York, 2003. The full version is available on the Cryptology ePrint Archive 2002/192.
9. C. Dwork, A. Goldberg, and M. Naor. On Memory-Bound Functions for Fighting Spam. In *Crypto '03*, LNCS 2729, pages 426–444. Springer-Verlag, Berlin, 2003.
10. IEEE Standard 1363.2 Study Group. Password-Based Public-Key Cryptography.
Available from <http://grouper.ieee.org/groups/1363/passwdPK>.
11. A. Juels and J. Brainard. Client Puzzles: A Cryptographic Defense Against Connection Depletion Attacks. In *Proc. of NDSS '99*, pages 151–165, 1999.
12. M. Luby and Ch. Rackoff. How to Construct Pseudorandom Permutations from Pseudorandom Functions. *SIAM Journal of Computing*, 17(2):373–386, 1988.
13. V. Shoup. OAEP Reconsidered. *Journal of Cryptology*, 15(4):223–249, September 2002.

A The Masked Diffie-Hellman Key Exchange Scheme

A.1 Description of the Scheme

In this scheme, as before, the arithmetic is in a finite cyclic group $\mathbb{G} = \langle g \rangle$ of order a ℓ -bit prime number q , where the operation is denoted multiplicatively. A hash function from $\{0, 1\}^*$

to $\{0, 1\}^\ell$ is denoted $\mathcal{H}$. While $\mathcal{G}$ denotes a full-domain hash function from $\{0, 1\}^*$ into $\mathbb{G}$. As illustrated on Figure 5 (with an honest execution of the MDHKE protocol), the protocol runs between two parties A and S , and the session-key space $\mathbf{SK}$ associated to this protocol is $\{0, 1\}^\ell$ equipped with a uniform distribution. It works as follows. The client chooses at random a private random exponent x and computes its Diffie-Hellman public value g^x . The client encrypts this latter value using a mask generation function, as the product of a Diffie-Hellman value with a full-domain hash of the password, and sends it to the server. The server in turn chooses at random a private random exponent y and computes its Diffie-Hellman public value g^y which it encrypts using the mask generation function. The client (resp. server) then decrypts the flow it has received and computes the session key.

A.2 Square Diffie-Hellman Assumption

A variant to the classical computational Diffie-Hellman problem is the particular case where $y = x$: the computational square Diffie-Hellman problem, against which the success of any adversary is

$$\text{Succ}_{g, \mathbb{G}}^{\text{csqdh}}(\Delta) = \Pr_x[\Delta(g^x) = g^{x^2}].$$

The same way as before, one defines $\text{Succ}_{g, \mathbb{G}}^{\text{csqdh}}(t)$.

Granted $4xy = (x + y)^2 - (x - y)^2$, one gets

$$\text{CDH}_{g, \mathbb{G}}(X, Y) = \left(\frac{\text{CSQDH}_{g, \mathbb{G}}(XY)}{\text{CSQDH}_{g, \mathbb{G}}(X/Y)} \right)^{1/4}.$$

The random self-reducibility $\text{CSQDH}_{g, \mathbb{G}}(X^r)\text{CSQDH}_{g, \mathbb{G}}(X)^{r^2}$, since $\mathbb{G}$ is a prime order group, leads to

$$\text{Succ}_{g, \mathbb{G}}^{\text{cdh}}(2t + 5\tau_e + 2\tau_m + 3\tau_i) \geq \left(\text{Succ}_{g, \mathbb{G}}^{\text{csqdh}}(t) \right)^2,$$

where τ_e denotes the time for an exponentiation in $\mathbb{G}$, τ_m the time for a multiplication in $\mathbb{G}$, and τ_i the time for an inversion in $\mathbb{Z}_q^*$. Therefore, the square Diffie-Hellman problem is as hard as the basic computational Diffie-Hellman problem.

A.3 Security Result

In this section, we assert that under the intractability of the above square Diffie-Hellman problem, the EKE protocol, with a one-time pad (hence named MDHKE), securely distributes session keys: the key is semantically secure.

Theorem 4 (AKE Security). *Let us consider the above protocol MDHKE, over a group of prime order q , where Password is a dictionary equipped with the distribution $\mathcal{D}_{pw}$. Let $\mathcal{A}$ be an adversary against the AKE security within a time bound t , with less than q_s active interactions with the parties (Send-queries) and q_p passive eavesdroppings (Execute-queries), and, asking q_g and q_h hash queries to $\mathcal{G}$ and $\mathcal{H}$ respectively. Then we have*

$$\text{Adv}_{\text{mdhke}}^{\text{ake}}(\mathcal{A}) \leq 2 \times \mathcal{D}_{pw}(q_s) + 4q_h^2 \times \text{Succ}_{g, \mathbb{G}}^{\text{csqdh}}(t + 3\tau_e) + \frac{(q_p + q_s)^2 + 3q_g^2}{q},$$

where τ_e denotes the computational time for an exponentiation in $\mathbb{G}$.

Proof. This proof is very similar to the previous one. However, in this new proof, we are just interested in the event S_n , which occurs if $b = b'$, where b is the bit involved in the **Test**-query, and b' is the output of the AKE-adversary. Let us thus briefly present the sequence of games.

Game $\mathbf{G}_0$: This is the real protocol, in the random-oracle model. By definition, we have

$$\text{Adv}_{\text{mdhke}}^{\text{ake}}(\mathcal{A}) = 2 \Pr[S_0] - 1. \quad (8)$$

Game $\mathbf{G}_1$: In this game, we simulate the hash oracles ($\mathcal{G}$ and $\mathcal{H}$, but also an additional hash

$\mathcal{G}$ and $\mathcal{H}$ oracles	For a hash-query $\mathcal{H}(q)$ (resp. $\mathcal{H}'(q)$), such that a record (q, r) appears in $\Lambda_{\mathcal{H}}$ (resp. $\Lambda_{\mathcal{H}'}$), the answer is r . Otherwise one chooses a random element $r \in \{0, 1\}^\ell$, answers with it, and adds the record (q, r) to $\Lambda_{\mathcal{H}}$ (resp. $\Lambda_{\mathcal{H}'}$).
	For a hash-query $\mathcal{G}(q)$ such that a record $(q, r, \star)$ appears in $\Lambda_{\mathcal{G}}$, the answer is r . Otherwise the answer r is defined according to the following rule:
	► Rule $\mathcal{G}^{(1)}$ Choose a random element $r \in \mathbb{G}$. The record $(q, r, \perp)$ is added to $\Lambda_{\mathcal{G}}$.
	Note: the third component of the elements of this list will be explained later.

Fig. 6. Simulation of the MDHKE protocol

function $\mathcal{H}' : \{0, 1\}^* \rightarrow \{0, 1\}^\ell$ that will appear in the Game $\mathbf{G}_3$) as usual by maintaining hash lists $\Lambda_{\mathcal{G}}$, $\Lambda_{\mathcal{H}}$ and $\Lambda_{\mathcal{H}'}$ (see Figure 6). We also simulate all the instances, as the real players would do, for the **Send**-queries and for the **Execute**, **Reveal** and **Test**-queries (see Figure 7). From this simulation, we easily see that the game is perfectly indistinguishable from the real attack.

Game $\mathbf{G}_2$: As in the previous proof, for an easier analysis in the following, we cancel games in which some collisions appear:

- collisions on the transcripts $((A, X^*), (S, Y^*))$;
- collisions on the output of $\mathcal{G}$.

Both probabilities are bounded by the birthday paradox:

$$\Pr[\text{Coll}_2] \leq \frac{(q_p + q_s)^2}{2q} + \frac{q_g^2}{2q}. \quad (9)$$

Game $\mathbf{G}_3$: In this game, we do not compute the session key sk using the oracle $\mathcal{H}$, but using the private oracle $\mathcal{H}'$ so that the value sk is not only completely independent from $\mathcal{H}$, but also independent from pw and thus from both K_A and K_S . We reach this aim by using the following rules:

► **Rule A3/S3⁽³⁾**

- | Compute the session key $sk_{A/S} = \mathcal{H}'(A \| S \| X^* \| Y^*)$.

Since we do no longer need to compute the values K_A and K_S , we can also simplify the second rules:

► **Rule A2/S2⁽³⁾**

- | Do nothing.

Send-queries to A	We answer to the Send-queries to an A-instance as follows: – A Send(A^i, Start)-query is processed according to the following rule: ► Rule A1⁽¹⁾ - Choose a random exponent $\theta \in \mathbb{Z}_q$, compute $X = g^\theta$ and $X^* = X \times \text{PW}$. Then the query is answered with (A, X^*), and the instance goes to an expecting state. – If the instance A^i is in an expecting state, a query Send($A^i, (S, Y^*)$) is processed by computing the session key. We apply the following rules: ► Rule A2⁽¹⁾ - Compute $Y = Y^* / \text{PW}$ and $K_A = Y^\theta$. ► Rule A3⁽¹⁾ - Compute the session key $sk_A = \mathcal{H}(A \ S \ X^* \ Y^* \ \text{PW} \ K_A)$. Finally the instance accepts.
Send-queries to S	We answer to the Send-queries to a S-instance as follows: – A Send($S^j, (A, X^*)$)-query is processed according to the following rules: ► Rule S1⁽¹⁾ - Choose a random exponent $\varphi \in \mathbb{Z}_q$, compute $Y = g^\varphi$ and $Y^* = Y \times \mathcal{G}(pw)$. Then the query is answered with (S, Y^*), and the instance applies the following rules. ► Rule S2⁽¹⁾ - Compute $X = X^* / \text{PW}$ and $K_S = X^\varphi$. ► Rule S3⁽¹⁾ - Compute the session key $sk_S = \mathcal{H}(A \ S \ X^* \ Y^* \ \text{PW} \ K_S)$. Finally, the instance accepts.
Other queries	An Execute(A^i, S^j)-query is processed using successively the above simulations of the Send-queries: $(A, X^*) \leftarrow \text{Send}(A^i, \text{Start})$ and $(S, Y^*) \leftarrow \text{Send}(S^j, (A, X^*))$, and outputting the transcript $((A, X^*), (S, Y^*))$. A Reveal(U)-query returns the session key (sk_A or sk_S) computed by the instance I (if the latter has accepted). A Test(U)-query first gets sk from Reveal(U), and flips a coin b. If $b = 1$, we return the value of the session key sk, otherwise we return a random value drawn from $\{0, 1\}^\ell$.

Fig. 7. Simulation of the MDHKE protocol

The games $\mathbf{G}_3$ and $\mathbf{G}_2$ are indistinguishable unless the following event **AskH** occurs: $\mathcal{A}$ queries the hash function $\mathcal{H}$ on $A \| S \| X^* \| Y^* \| \text{PW} \| K_A$ or on $A \| S \| X^* \| Y^* \| \text{PW} \| K_S$, for some execution transcript $((A, X^*), (S, Y^*))$. This means that, for *some transcript* $((A, X^*), (S, Y^*))$, which number is upper-bounded by $q_s + q_p$, the tuple $(X^*, Y^*, \text{PW}, \text{CDH}_{g, \mathbb{G}}(X^* / \text{PW}, Y^* / \text{PW}))$ lies in the list $\Lambda_{\mathcal{H}}$.

On the other hand, the session key is computed with a random oracle that is private to the simulator, then one can remark that it cannot be distinguished by the adversary unless the same transcript $((A, X^*), (S, Y^*))$ appeared in another session, for which a **Reveal**-query has been asked (which event has been excluded in the previous game):

$$\Pr[\mathbf{S}_3] = \frac{1}{2}. \quad (10)$$

Actually, one does not need the password for the simulation either: we can formally simplify again some rules but thus without modifying anything w.r.t. the probabilities:

► **Rule A1**⁽³⁾

- | Choose a random element $x \in \mathbb{Z}_q$ and compute $X^* = g^x$.

► **Rule S1**⁽³⁾

Choose a random element $y \in \mathbb{Z}_q$ and compute $Y^* = g^y$.

Game G₄: In order to evaluate the probability of event AskH, let us modify the simulation of the oracle $\mathcal{G}$, with a random $Q \in \mathbb{G} \setminus \{1\}$ (which is thus a generator of $\mathbb{G}$, since the latter has a prime order q). The simulation introduces values in the third component of the elements of $\Lambda_{\mathcal{G}}$, but does not use it. It would let the probabilities unchanged, but we exclude the case $\text{PW} = 1$:

► **Rule G**⁽⁴⁾

Choose a random element $k \in \mathbb{Z}_q^*$ and compute $r = Q^{-k}$. The record (q, r, k) is added to $\Lambda_{\mathcal{G}}$.

Since we just exclude $k = 0$, we have:

$$|\Pr[\text{AskH}_4] - \Pr[\text{AskH}_3]| \leq \frac{qg}{q}. \quad (11)$$

Game G₅: It is now possible to evaluate the probability of the event AskH. Indeed, one can remark that the password is never used during the simulation. It thus does not need to be chosen in advance, but at the very end only, to check whether the event AskH happened or not. To make this evaluation easier, we cancel the games wherein for some pair $(X^*, Y^*) \in \mathbb{G}^2$, involved in a communication, there are two elements $\text{PW} \neq 1$ such that the tuple $(X^*, Y^*, \text{PW}, \text{CDH}_{g, \mathbb{G}}(X^*/\text{PW}, Y^*/\text{PW}))$ is in $\Lambda_{\mathcal{H}}$ (which event is denoted CollH₅). Hopefully, event CollH₅ can be upper-bounded, granted the following Lemma:

Lemma 5. *For any pair $(X^*, Y^*) \in \mathbb{G}^2$, involved in a communication, there is at most one element $\text{PW} \neq 1$ such that $(X^*, Y^*, \text{PW}, \text{CDH}_{g, \mathbb{G}}(X^*/\text{PW}, Y^*/\text{PW}))$ in $\Lambda_{\mathcal{H}}$, unless one can solve the square Diffie-Hellman problem:*

$$\Pr[\text{CollH}_5] \leq q_h^2 \times \text{Succ}_{g, \mathbb{G}}^{\text{csqdh}}(t + 3\tau_e). \quad (12)$$

Proof. Assume there exist $(X^* = g^x, Y^* = g^y) \in \mathbb{G}^2$ involved in a communication, $\text{PW}_0 = Q^{-k_0} \neq 1$, and $\text{PW}_1 = Q^{-k_1} \neq 1$ such that the tuple

$$(X^*, Y^*, \text{PW}_i, Z_i = \text{CDH}_{g, \mathbb{G}}(X^*/\text{PW}_i, Y^*/\text{PW}_i))$$

is in $\Lambda_{\mathcal{H}}$, for $i = 0, 1$. Then,

$$\begin{aligned} Z_i &= \text{CDH}_{g, \mathbb{G}}(X^* \times Q^{k_i}, Y^* \times Q^{k_i}) \\ &= \text{CDH}_{g, \mathbb{G}}(X^*, Y^*) \times \text{CDH}_{g, \mathbb{G}}(X^* \times Y^*, Q)^{k_i} \times \text{CSQDH}_{g, \mathbb{G}}(Q)^{k_i^2} \\ Z_1^{k_0}/Z_0^{k_1} &= \text{CDH}_{g, \mathbb{G}}(X^*, Y^*)^{k_0-k_1} \times \text{CSQDH}_{g, \mathbb{G}}(Q)^{k_0 k_1 (k_1 - k_0)}. \end{aligned}$$

Since $(X^*, Y^*) \in \mathbb{G}^2$ has been involved in a communication (either from Send-queries or an Execute-query), one of X^*g^x or $Y^* = g^y$, has been simulated: at least one of x or y is known, and thus $Z = \text{CDH}_{g, \mathbb{G}}(X^*, Y^*)$ is so too. As a consequence,

$$\text{CSQDH}_{g, \mathbb{G}}(Q) = \left((Z_1/Z)^{k_0} \times (Z_0/Z)^{-k_1} \right)^u,$$

where u is the inverse of $k_0 k_1 (k_1 - k_0)$ in $\mathbb{Z}_q$. The latter exists since $\text{PW}_1 \neq \text{PW}_2$, and they are both distinct of 1. By guessing the two queries asked to $\mathcal{H}$, one concludes the proof. $\square$

For a more convenient analysis, we can split the event AskH in two disjoint sub-cases:

1. **AskHPassive**, where the transcript $((A, X^*), (S, Y^*))$ involved in the crucial $\mathcal{H}$ -query comes as an answer from an **Execute**-query;
2. **AskHActive**, the other cases.

About the active case (the event **AskHActive**₅), the above Lemma 5 applied to games where the event **CollH**₅ did not happen only states that for each pair (X^*, Y^*) involved in an active transcript, there is at most one **PW** such that the corresponding tuple is in $\Lambda_{\mathcal{H}}$. Since we also excluded games with collisions on $\mathcal{G}$, the latter is an injection and thus the uniqueness of **PW** translates to the uniqueness of pw such that $\text{PW} = \mathcal{G}(pw)$:

$$\Pr[\text{AskHActive}_5] \leq \mathcal{D}_{pw}(q_s). \quad (13)$$

Moreover, in the particular case of passive transcripts, one can state a stronger result:

Lemma 6. *For any pair $(X^*, Y^*) \in \mathbb{G}^2$, involved in a passive transcript, there is no element $\text{PW} \neq 1$ such that $(X^*, Y^*, \text{PW}, \text{CDH}_{g, \mathbb{G}}(X^*/\text{PW}, Y^*/\text{PW}))$ in $\Lambda_{\mathcal{H}}$, unless one can solve the square Diffie-Hellman problem:*

$$\Pr[\text{AskHPassive}_5] \leq q_h \times \text{Succ}_{g, \mathbb{G}}^{\text{csqdh}}(t + 3\tau_e). \quad (14)$$

Proof. Assume there exist $(X^* = g^x, Y^* = g^y) \in \mathbb{G}^2$ involved in a passive transcript, and $\text{PW} = Q^{-k} \neq 1$ such that the tuple $(X^*, Y^*, \text{PW}, Z = \text{CDH}_{g, \mathbb{G}}(X^*/\text{PW}, Y^*/\text{PW}))$ is in $\Lambda_{\mathcal{H}}$. Then, as above,

$$Z = g^{xy} \times Q^{k(x+y)} \times \text{CSQDH}_{g, \mathbb{G}}(Q)^{k^2}.$$

As a consequence,

$$\text{CSQDH}_{g, \mathbb{G}}(Q) = (Z \times \text{PW}^{x+y}/g^{xy})^u,$$

where u is the inverse of k^2 in $\mathbb{Z}_q$. By guessing the query asked to $\mathcal{H}$, one concludes the proof. $\square$

As a conclusion,

$$\Pr[\text{AskH}_5] \leq q_h \times \text{Succ}_{g, \mathbb{G}}^{\text{csqdh}}(t + 3\tau_e) + \mathcal{D}_{pw}(q_s). \quad (15)$$

Combining all the above equations, one gets

$$\text{Adv}_{\text{mdhke}}^{\text{ake}}(\mathcal{A}) \leq 2 \times \left(\mathcal{D}_{pw}(q_s) + q_h \times \text{Succ}_{g, \mathbb{G}}^{\text{csqdh}}(t + 3\tau_e) + q_h^2 \times \text{Succ}_{g, \mathbb{G}}^{\text{csqdh}}(t + 3\tau_e) \right) \left(+ \frac{q_g}{q} + \frac{q_g^2}{2q} + \frac{(q_p + q_s)^2}{2q} \right).$$

$\square$